University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 2

Date: October 14, 2024

Group Number: 75

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Wendy Phung	76185974	z1l1m	ptat1907@gmail.com
Nilasha Majumdar	55099899	u7n5u	nilashamajumdar2021@gmail.com
Victoria Baek	40749855	c4g4x	vickybaek20@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

2. **Project Summary:**

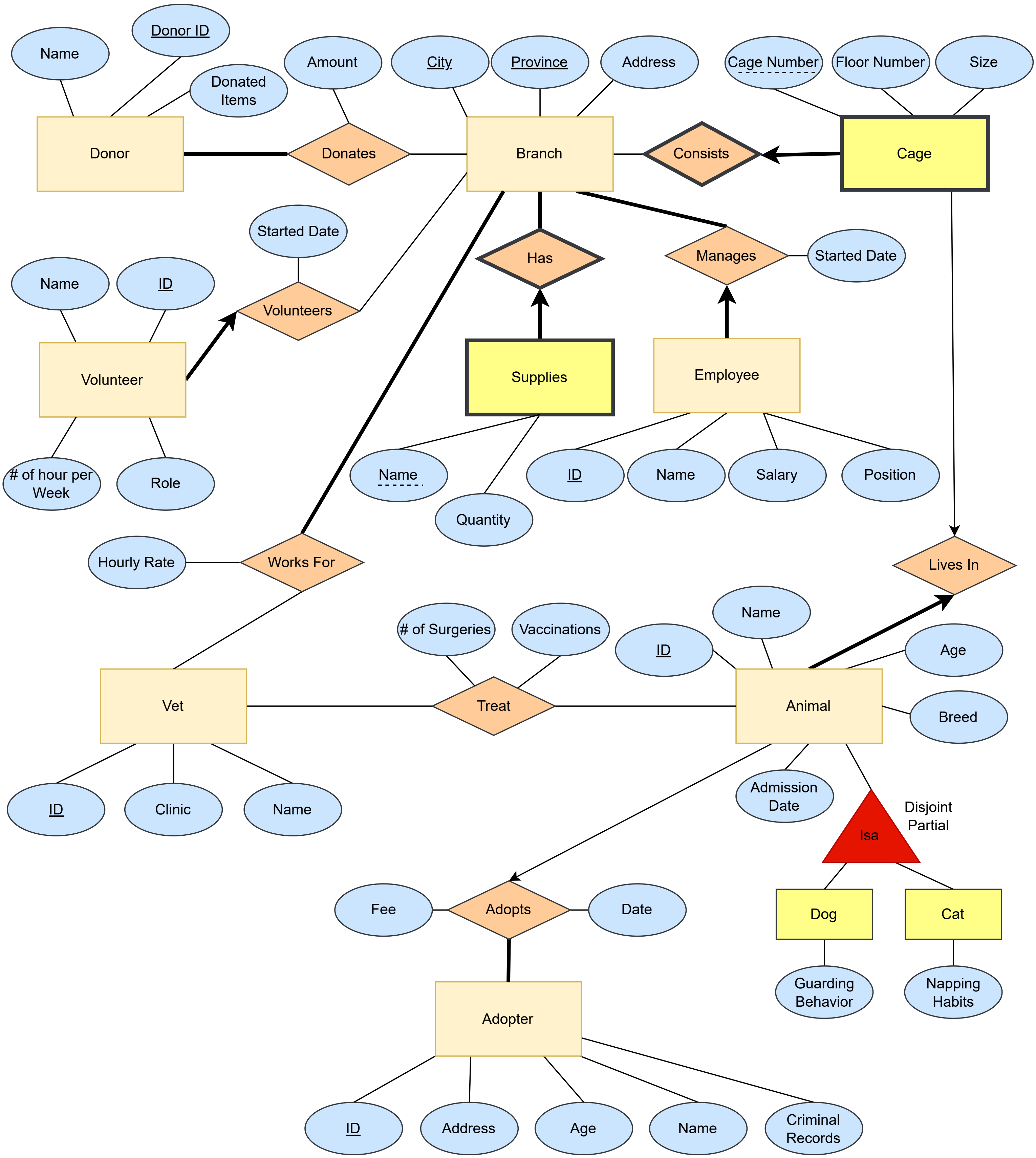
   This project focuses on developing a database system for animal shelter management, which includes tracking animal profiles, adoption processes, shelter operations, and donor and volunteer information. The system models various aspects of shelter management, such as branch locations, animal assignments to cages, veterinary services, and supplies management. By efficiently organizing and updating records, the database ensures smooth operations and effective care for the animals in the shelter.

3. **ER Diagram**

   This ER diagram is the same as our Milestone 1 submission. There were no suggestions for modifications provided by our project mentor, so we have not made any changes. Moreover, after reviewing the design and considering future project requirements, we believe our diagram adequately supports our system's functionality.

   The following names have been changed slightly from the ER diagram:
   For all the table names, the first letter is capitalized.
   For all the attribute names, the letters have been changed to lower case (except for the word "ID").

   - For the Donor entity: Donated items → donated_item
   - For the Volunteer entity: ID → volunteer_ID
   - For the Volunteer entity: # of hour per Week → hours_per_week
   - For the Vet entity: ID → vet_ID
   - Works For → WorkFor
   - For the WorkFor relationship set: Hourly Rate → hourly_rate
   - For the Employee entity: ID → employee_ID
   - For the Manages relationship: Started Date → started_date
   - For the Cage entity: Cage Number → cage_number
   - For the Cage entity: Floor Number → floor_number
   - For the Animal entity: ID → animal_ID
   - For the Animal entity: Admission Date → admission_date
   - For the Dog and Cat entities: Guarding Behavior → guarding_behavior and Napping Habits → napping_habits
   - For the Adopts relationship set: Fee, Date → adiption_fee, adoption_date
   - For the Adopter entity: ID → adopter_ID
   - For the Adopter entity: criminal records → criminal_record (removed s)
   - For the Treat relationship set: # of Surgeries → number_of_surgeries


Name
Donor ID
Donated Items
Donor
Amount
Donates
City
Province
Address
Branch
Consists
Cage Number
Floor Number
Size
Cage
Started Date
Name
ID
Volunteers
Volunteer
# of hour per Week
Role
Has
Supplies
Manages
Started Date
Employee
Name
Quantity
ID
Name
Salary
Position
Hourly Rate
Works For
Lives In
# of Surgeries
Vaccinations
Name
ID
Age
Vet
Treat
Animal
Breed
ID
Clinic
Name
Admission Date
Isa
Disjoint Partial
Fee
Adopts
Date
Dog
Cat
Guarding Behavior
Napping Habits
Adopter
ID
Address
Age
Name
Criminal Records

4. Relational Schema:

```
Donor ( donor_ID          INTEGER,
        name              VARCHAR(20),
        donated_item      VARCHAR(20),
        PRIMARY KEY (donor_ID))      -- also CANDIDATE KEY

Donate ( donor_ID         INTEGER,
         branch_city      VARCHAR(20),
         branch_province  VARCHAR(20),
         amount           DECIMAL(8, 2),
         PRIMARY KEY (donor_ID, branch_city, branch_province),
         FOREIGN KEY (donor_ID) REFERENCES Donor(donor_ID),
         FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city, province))

Branch ( city             VARCHAR(20),
         province         VARCHAR(20),
         address          VARCHAR(50),
         PRIMARY KEY (city, province))

Volunteer ( volunteer_ID      INTEGER,
            name              VARCHAR(20),
            hours_per_week    INTEGER,
            role              VARCHAR(20),
            started_date      DATE,
            branch_city       VARCHAR (20) NOT NULL,
            branch_province   VARCHAR(20) NOT NULL,
            PRIMARY KEY (volunteer_ID),   -- also CANDIDATE KEY
            FOREIGN KEY (branch_city, branch_province) REFERENCES
                  Branch (city, province) ON DELETE NO ACTION
                                          ON UPDATE CASCADE)

Vet ( vet_ID   INTEGER,
      name     VARCHAR(20),
      clinic   VARCHAR(50),
      PRIMARY KEY (vet_ID))    -- also CANDIDATE KEY
```

WorkFor (vet_ID INTEGER,
branch_city VARCHAR(20),
branch_province VARCHAR(20),
hourly_rate DECIMAL(4,2),
PRIMARY KEY (vet_ID, branch_city, branch_province) -- also **CANDIDATE KEY**
FOREIGN KEY (vet_ID) **REFERENCES** Vet (vet_ID),
FOREIGN KEY (branch_city, branch_province) **REFERENCES** Branch (city, province))

Supplies (name VARCHAR(50),
quantity INTEGER,
branch_city VARCHAR (20),
branch_province VARCHAR (20),
PRIMARY KEY (name, branch_city, branch_province) -- also **CANDIDATE KEY**
FOREIGN KEY (branch_city, branch_province) **REFERENCES**
Branch (city, province), **ON DELETE CASCADE**)

EmployeeManages (employee_ID INTEGER,
name VARCHAR(20),
salary INTEGER,
position VARCHAR(20),
started_date DATE,
branch_city VARCHAR (20) NOT NULL,
branch_province VARCHAR(20) NOT NULL,
PRIMARY KEY (employee_ID), -- also **CANDIDATE KEY**
FOREIGN KEY (branch_city, branch_province) **REFERENCES**
Branch (city, province) **ON DELETE NO ACTION ON UPDATE CASCADE**)

Cage (cage_number INTEGER,
floor_number INTEGER,
size VARCHAR (20),
branch_city VARCHAR (20),
branch_province VARCHAR (20),
PRIMARY KEY (cage_number, branch_city, branch_province), -- also **CANDIDATE KEY**
FOREIGN KEY (branch_city, branch_province) **REFERENCES**
Branch (city, province), **ON DELETE CASCADE**)

```
Animal ( animal_ID          INTEGER,
         name               VARCHAR (20),
         age                INTEGER,
         breed              VARCHAR (20),
         admission_date     DATE,
         adoption_fee       INTEGER,
         adoption_date      DATE,
         adopter_ID         INTEGER,
         cage_number        INTEGER NOT NULL,
         branch_city        VARCHAR (20) NOT NULL,
         branch_province    VARCHAR (20) NOT NULL,
         UNIQUE (cage_number, branch_city, branch_province),
         PRIMARY KEY (animal_ID),          -- also CANDIDATE KEY
         FOREIGN KEY (adopter_ID) REFERENCES Adopter (adopter_ID),
         FOREIGN KEY (cage_number) REFERENCES Cage (cage_number),
         FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city, province))

Dog ( animal_ID             INTEGER,
      guarding_behavior     VARCHAR (20),
      PRIMARY KEY (animal_ID),     -- also CANDIDATE KEY
      FOREIGN KEY (animal_ID) REFERENCES Animal (animal_ID))

Cat ( animal_ID             INTEGER,
      napping_habits        VARCHAR (20),
      PRIMARY KEY (animal_ID),     -- also CANDIDATE KEY
      FOREIGN KEY (animal_ID) REFERENCES Animal (animal_ID))

Treat ( vet_ID                  INTEGER,
        animal_ID               INTEGER,
        number_of_surgeries     INTEGER,
        vaccination             BOOLEAN,
        PRIMARY KEY (vet_ID, animal_ID)      -- also CANDIDATE KEY
        FOREIGN KEY (vet_ID) REFERENCES Vet (vet_ID),
        FOREIGN KEY (animal_ID) REFERENCES Animal (animal_ID))
```

```
Adopter (adopter_ID       INTEGER,
        name              VARCHAR (20),
        age               INTEGER,
        address           VARCHAR (50),
        criminal_record   VARCHAR (20),
        PRIMARY KEY (adopter_ID))
```

5. Functional Dependencies:

For each relation, the keys are underlined, and foreign keys are bold.

Donor (donor_ID, name, donated_Item)
- donor_ID → name, donated_Item

Donate (**donor_ID, branch_city**, **branch_province**, amount)
- donor_ID, branch_city, branch_province → amount

Branch (**city**, **province**, address)
- city, province → address

Volunteer (volunteer_ID, name, hours_per_week, role, started_date, **branch_city**, **branch_province**)
- volunteer_ID → name, hours_per_week, role, started_date, branch_city, branch_province
- role → hours_per_week

Vet (vet_ID, name, clinic)
- vet_ID → name, clinic

WorkFor (**vet_ID**, **branch_city**, **branch_province**, hourly_rate)
- vet_ID, branch_city, branch_province → hourly_rate

Supplies (name, **branch_city**, **branch_province**, quantity)
- name, branch_city, branch_province → quantity

EmployeeManages (employee_ID, **branch_city**, **branch_province**, name, salary, position, started_date)
- employee_ID → name, salary, position, started_date, branch_city, branch_province
- position → salary

Cage (cage_number, **branch_city**, **branch_province**, floor_number, size)

- cage_Number, branch_city, branch_province → floor_number, size
- floor_number → size

Animal (animal_ID, name, age, breed, admission_date, adoption_fee, adoption_date, **adopter_ID**, **cage_number**, **branch_city**, **branch_province**)

- animal_ID → name, age, breed, admission_date, adoption_fee, adoption_date, adopter_ID, cage_number, branch_city, branch_province
- breed, age → adoption_fee

Dog (**animal_ID**, guarding_behavior)

- animal_ID → guarding_behavior

Cat (**animal_ID**, napping_habits)

- animal_ID → napping_habits

Treat (**vet_ID**, **animal_ID**, number_of_surgeries, vaccination)

- vet_ID, animal_ID → number_of_surgeries, vaccination

Adopter (adopter_ID, name, age, address, criminal_record)

- adopter_ID → name, age, address, criminal_record

6. **Normalization:**

The relations that violated 3NF are Volunteer, EmployeeManages, Cage, and Animal. The followings are the 3NF decompositions:

1. Volunteer (<u>volunteer_ID</u>, name, hours_per_week, role, started_Date, **branch_city**, **<u>branch_province</u>**)
   - volunteer_ID → name, hours_per_week, role, started_date, branch_city, branch_province
   - role → hours_per_week

   The closures are:
   - {volunteer_ID}+ = {name, hours_per_week, role, started_date, branch_city, branch_province}
   - {role}+ = {role, hours_per_week}

   Volunteer_ID is the minimal key. Role is not a superkey and hours_per_week is not a part of the minimal key.
   - Then, role → hours_per_week violates 3NF. This table is not in 3NF.

   Minimal cover:
   - volunteer_ID → name
   - volunteer_ID → hours_per_week
   - volunteer_ID → role
   - volunteer_ID → started_date
   - volunteer_ID → branch_city
   - volunteer_ID → branch_province
   - role → hours_per_week

   From the minimal cover above, only role → hours_per_week violates BCNF.

   Decomposition into 3NF using the lossless join method:

   Decompose the relation Volunteer on role → hours_per_week and we get two new relations:

Volunteer (volunteer_ID, name, **role**, started_date, **branch_city**, **branch_province**)

Role(role, hours_per_week)

2. EmployeeManages (employee_ID, **branch_city**, **branch_province**, name, salary, position, started_date)

- employee_ID → name, salary, position, started_date, branch_city, branch_province
- position → salary

The closures are:

- {employee_ID}+ = {employee_ID, name, salary, position, started_date, branch_city, branch_province}
- {position}+ = {position, salary}

employee_ID is the minimal key. Position is not a superkey and salary is not a part of the minimal key.

Then, position → salary violates 3NF. This table is not in 3NF.

Minimal cover:

- employee_ID → name
- employee_ID → salary
- employee_ID → position
- employee_ID → started_date
- employee_ID → branch_city
- employee_ID → branch_province
- position → salary

From the minimal cover above, only position → salary violates BCNF.

Decomposition into 3NF using the lossless join method:

Decompose the relation EmployeeManages on position → salary and we get two new relations:

EmployeeManages (employee_ID, **banch_city**, **branch_province**, name, **position**, started_date)

Position (position, salary)

3. Cage (cage_number, **branch_city**, **branch_province**, floor_number, size)
   - cage_number, branch_city, branch_province → floor_number, size
   - floor_number → size

The closures are:
- {cage_number, branch_city, branch_province}+ = {cage_number, branch_city, branch_province, floor_number, size}
- {floor_number}+ = {floor_number, size}

cage_number, branch_city, branch_province is the minimal key. floor_number is not a superkey and size is not a part of the minimal key.

Then, floor_number → size violates 3NF. This table is not in 3NF.

Minimal cover:
- cage_number, branch_city, branch_province → floor_number
- cage_number, branch_city, branch_province → size
- floor_number → size

From the minimal cover above, only floor_number → size violates BCNF.

Decomposition into 3NF using the lossless join method:

Decompose the relation Cage on floor_number → size and we get two new relations:

Cage (cage_number, **branch_city**, **branch_province**, **floor_number**)

FloorNumber(floor_number, size)

4. Animal (animal_ID, name, age, breed, admission_date, adoption_fee, adoption_date, **adopter_ID**, **cage_number**, **branch_city**, **branch_province**)
   - animal_ID → name, age, breed, admission_date, adoption_fee, adoption_date, adopter_ID, cage_number, branch_city, branch_province
   - breed, age → adoption_fee

   The closures are:
   - {animal_ID}+ = {animal_ID, name, age, breed, admission_date, adoption_fee, adoption_date, adopter_ID, cage_number, branch_city, branch_province}
   - {breed, age}+ = {adoption_fee}

   animal_ID is the minimal key. {breed, age} is not a superkey and doption_fee is not a part of the minimal key.

   Then, breed, age → adoption_fee violates 3NF. This table is not in 3NF.

   Minimal covers:
   - animal_ID → name
   - animal_ID → age
   - animal_ID → breed
   - animal_ID → admission_date
   - animal_ID → admission_fee
   - animal_ID → adoption_date
   - animal_ID → adopter_ID
   - animal_ID → aage_number
   - animal_ID → branch_city
   - animal_ID → branch_province
   - breed, age → ddoption_fee

   From the minimal cover above, only breed, age → adoption_fee violates BCNF.

   Decomposition into 3NF using the lossless join method:

   Decompose the relation Animal on breed, age → adoption_fee and we get two new relations:

Animal (animal_ID, name, **age**, **breed**, admission_date, adoption_date, **adopter_ID**, **cage_number**, **branch_city**, **branch_province**)

AdoptionFee (breed, age, adoption_fee)

Therefore, the final tables are:

```
Donor ( donor_ID           INTEGER,
        name               VARCHAR(20),
        donated_item       VARCHAR(20),
        PRIMARY KEY (donor_ID))      -- also CANDIDATE KEY

Donate ( donor_ID          INTEGER,
         branch_city       VARCHAR(20),
         branch_province   VARCHAR(20),
         amount            DECIMAL(8, 2),
         PRIMARY KEY (donor_ID, branch_city, branch_province),
         FOREIGN KEY (donor_ID) REFERENCES Donor(donor_ID),
         FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city, province))

Branch ( city              VARCHAR(20),
         province          VARCHAR(20),
         address           VARCHAR(50),
         PRIMARY KEY (city, province))

Volunteer ( volunteer_ID   INTEGER,
            name           VARCHAR(20),
            role           VARCHAR(20),
            started_date   DATE,
            branch_city    VARCHAR (20) NOT NULL,
            branch_province VARCHAR(20) NOT NULL,
            PRIMARY KEY (volunteer_ID),   -- also CANDIDATE KEY
            FOREIGN KEY (role) REFERENCES Role (role),
            FOREIGN KEY (branch_city, branch_province) REFERENCES
                Branch (city, province) ON DELETE NO ACTION ON UPDATE CASCADE)
```

```
Role ( role                 VARCHAR(20),
       hours_per_week       INTEGER,
       PRIMARY KEY (role))    -- also CANDIDATE KEY

Vet ( vet_ID   INTEGER,
      name     VARCHAR(20),
      clinic   VARCHAR(50),
      PRIMARY KEY (vet_ID))   -- also CANDIDATE KEY

WorkFor ( vet_ID                 INTEGER,
          branch_city            VARCHAR(20),
          branch_province        VARCHAR(20),
          hourly_rate            DECIMAL(4,2),
          PRIMARY KEY (vet_ID, branch_city, branch_province)     -- also CANDIDATE KEY
          FOREIGN KEY (vet_ID) REFERENCES Vet(vet_ID),
          FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city, province))

Supplies ( name                  VARCHAR(50),
           quantity              INTEGER,
           branch_city           VARCHAR (20),
           branch_province       VARCHAR (20),
           PRIMARY KEY (name, branch_city, branch_province)       -- also CANDIDATE KEY
           FOREIGN KEY (branch_city, branch_province) REFERENCES
                Branch (city, province), ON DELETE CASCADE)

EmployeeManages ( employee_ID        INTEGER,
                  name               VARCHAR(20),
                  position           VARCHAR(20),
                  started_date       DATE,
                  branch_city        VARCHAR (20) NOT NULL,
                  branch_province    VARCHAR(20) NOT NULL,
                  PRIMARY KEY (employee_ID),    -- also CANDIDATE KEY
                  FOREIGN KEY (position) REFERENCES Position (position),
                  FOREIGN KEY (branch_city, branch_province) REFERENCES
                       Branch (city, province) ON DELETE NO ACTION
                                               ON UPDATE CASCADE)
```

```
Position ( position      VARCHAR(20),
           salary        INTEGER,
           PRIMARY KEY (position)) -- also CANDIDATE KEY

Cage ( cage_number      INTEGER,
       floor_number     INTEGER,
       branch_city      VARCHAR (20),
       branch_province  VARCHAR (20),
       PRIMARY KEY (cage_number, branch_city, branch_province),     -- also CANDIDATE KEY
       FOREIGN KEY (floor_number) REFERENCES FloorNumber(floor_number),
       FOREIGN KEY (branch_city, branch_province) REFERENCES
           Branch (city, province), ON DELETE CASCADE)

FloorNumber ( floor_number     INTEGER,
              size             VARCHAR(20),
              PRIMARY KEY (floor_number))     -- also CANDIDATE KEY

Animal ( animal_ID            INTEGER,
         name                 VARCHAR (20),
         age                  INTEGER,
         breed                VARCHAR (20),
         admission date       DATE,
         adoption_date        DATE,
         adopter_ID           INTEGER,
         cage_number          INTEGER NOT NULL,
         branch_city          VARCHAR (20) NOT NULL,
         branch_province      VARCHAR (20) NOT NULL,
         UNIQUE (cage_number, branch_city, branch_province),
         PRIMARY KEY (animal_ID),              -- also CANDIDATE KEY
         FOREIGN KEY (breed, age) REFERENCES AdoptionFee(breed, age),
         FOREIGN KEY (adopter_ID) REFERENCES Adopter (adopter_ID),
         FOREIGN KEY (cage_number) REFERENCES Cage (cage_number),
         FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city, province))

AdoptionFee ( breed            VARCHAR(20),
              age              INTEGER,
              adoption_fee     INTEGER,
              PRIMARY KEY (breed, age))
```

Dog (animal_ID INTEGER,
guarding_behavior VARCHAR (20),
PRIMARY KEY (animal_ID), -- also **CANDIDATE KEY**
FOREIGN KEY (animal_ID) **REFERENCES** Animal (animal_ID))

Cat (animal_ID INTEGER,
napping_habits VARCHAR (20),
PRIMARY KEY (animal_ID), -- also **CANDIDATE KEY**
FOREIGN KEY (animal_ID) **REFERENCES** Animal (animal_ID))
Treat (vet_ID INTEGER,
animal_ID INTEGER,
number_of_surgeries INTEGER,
vaccination BOOLEAN,
PRIMARY KEY (vet_ID, animal_ID) -- also **CANDIDATE KEY**
FOREIGN KEY (vet_ID) **REFERENCES** Vet (vet_ID),
FOREIGN KEY (animal_ID) **REFERENCES** Animal (animal_ID))

Adopter (adopter_ID INTEGER,
name VARCHAR (20),
age INTEGER,
address VARCHAR (50),
criminal_record VARCHAR (20),
PRIMARY KEY (adopter_ID))

7. SQL DDL Statements:

```
CREATE TABLE Donor (donor_ID INTEGER,
                    name VARCHAR(20),
                    donated_item VARCHAR(20),
                    PRIMARY KEY (donor_ID));
```

```
CREATE TABLE Donate (donor_ID INTEGER,
                    branch_city VARCHAR(20),
                    branch_province VARCHAR(20),
                    amount DECIMAL(8,2),
                    PRIMARY KEY (donor_ID, branch_city, branch_province),
                     FOREIGN KEY (donor_ID) REFERENCES Donor(donor_ID),
                     FOREIGN KEY (branch_city, branch_province)
                      REFERENCES Branch(city, province));
```

Branch(**<u>City</u>**, **<u>Province</u>**, Address)

```
CREATE TABLE Branch (city VARCHAR(20),
                    province VARCHAR(20),
                    address VARCHAR(50),
                    PRIMARY KEY (city, province));
```

Volunteer(<u>Volunteer_ID</u>, Name, **Role**, Started_Date, **Branch_City**, **Branch_Province**)

```
CREATE TABLE Volunteer (volunteer_ID INTEGER,
                        name VARCHAR(20),
                        role VARCHAR(20),
                        started_date DATE,
                        branch_city VARCHAR(20) NOT NULL,
                        branch_province VARCHAR(20) NOT NULL,
                        PRIMARY KEY (volunteer_ID),
                        FOREIGN KEY (role) REFERENCES Role (role),
                        FOREIGN KEY (branch_city, branch_province)
                          REFERENCES Branch (city, province)
                          ON DELETE NO ACTION ON UPDATE CASCADE);
```

Role(<u>Role</u>, Hours_per_week)

```
CREATE TABLE Role (role VARCHAR(20),
                  hours_per_week INTEGER,
                  PRIMARY KEY (role));
```

Vet(Vet_ID, Name, Clinic)

```
CREATE TABLE Vet (vet_ID INTEGER,
                  name VARCHAR(20),
                  clinic VARCHAR(50),
                  PRIMARY KEY (vet_ID));
```

WorkFor(**Vet_ID**, **Branch_City**, **Branch_Province**, Hourly_Rate)

```
CREATE TABLE WorkFor (vet_ID INTEGER,
                      branch_city VARCHAR(20),
                      branch_province VARCHAR(20),
                      hourly_rate DECIMAL(4,2),
                      PRIMARY KEY (vet_ID, branch_city, branch_province),
                      FOREIGN KEY (vet_ID) REFERENCES Vet(vet_ID),
                      FOREIGN KEY (branch_city, branch_province)
                        REFERENCES Branch (city, province));
```

Supplies(Name, **Branch_City**, **Branch_Province**, Quantity)

```
CREATE TABLE Supplies (name VARCHAR(50),
                      branch_city VARCHAR(20),
                      branch_province VARCHAR(20),
                      quantity INTEGER,
                      PRIMARY KEY (name, branch_city, branch_province),
                      FOREIGN KEY (branch_city, branch_province)
                        REFERENCES Branch(city, province)
                        ON DELETE CASCADE);
```

EmployeeManages(Employee_ID, **Branch_City**, **Branch_Province**, Name, **Position**, Started_Date)

```
CREATE TABLE EmployeeManages (employee_ID INTEGER,
                              branch_city VARCHAR(20) NOT NULL,
                              branch_province VARCHAR(20) NOT NULL,
                              name VARCHAR(20),
                              position VARCHAR(20),
                              started_date DATE,
                              PRIMARY KEY (employee_ID),
                              FOREIGN KEY (position)
                                REFERENCES Position(position),
                              FOREIGN KEY (branch_city, branch_province)
                                REFERENCES Branch (city, province)
                                ON DELETE NO ACTION ON UPDATE CASCADE);
```

Position(<u>Position</u>, Salary)

```
CREATE TABLE Position (position VARCHAR(20),
                       salary INTEGER,
                       PRIMARY KEY (position));
```

Cage(<u>Cage_Number</u>, **<u>Branch_City</u>**, **<u>Branch_Province</u>**, **Floor_Number**)

```
CREATE TABLE Cage (cage_number INTEGER,
                   branch_city VARCHAR(20),
                   branch_province VARCHAR(20),
                   floor_number INTEGER,
                   PRIMARY KEY (cage_number, branch_city, branch_province),
                   FOREIGN KEY (floor_number)
                     REFERENCES FloorNumber(floor_number),
                   FOREIGN KEY (branch_city, branch_province)
                     REFERENCES Branch(city, province) ON DELETE CASCADE);
```

FloorNumber(<u>Floor_Number</u>, Size)

```
CREATE TABLE FloorNumber (floor_number INTEGER,
                          size VARCHAR(20),
                          PRIMARY KEY (floor_number));
```

Animal(<u>Animal_ID</u>, Name, **Breed, Age**, Admission_Date, Adoption_Date, **Adopter_ID**, **Cage_Number**, **Branch_City**, **Branch_Province**)

```
CREATE TABLE Animal (animal_ID INTEGER,
                     name VARCHAR(20),
                     breed VARCHAR(20),
                     age INTEGER,
                     admission_date DATE,
                     adoption_date DATE,
                     adoper_ID INTEGER,
                     cage_number INTEGER NOT NULL,
                     branch_city VARCHAR(20) NOT NULL,
                     branch_province VARCHAR(20) NOT NULL,
                     UNIQUE (cage_number, branch_city, branch_province),
                     PRIMARY KEY (animal_ID),
                     FOREIGN KEY (breed, age)
                       REFERENCES AdoptionFee(breed, age),
                     FOREIGN KEY(adopter_ID) REFERENCES Adopter(adopter_ID),
                     FOREIGN KEY (cage_number) REFERENCES Cage(cage_number),
                     FOREIGN KEY (branch_city, branch_province)
                       REFERENCES Branch(city, province));
```

AdoptionFee(Breed, Age, Adoption_Fee)

```
CREATE TABLE AdoptionFee (breed VARCHAR(20),
                          age INTEGER,
                          adoption_fee INTEGER,
                          PRIMARY KEY (breed, age));
```

Dog(**Animal_ID**, Guarding_Behavior)

```
CREATE TABLE Dog (animal_ID INTEGER,
                  guarding_behavior VARCHAR(20),
                  PRIMARY KEY (animal_ID),
                  FOREIGN KEY (animal_ID) REFERENCES Animal(animal_ID));
```

Cat(**Animal_ID**, Napping_Habits)

```
CREATE TABLE Cat (animal_ID INTEGER,
                  napping_habits VARCHAR(20),
                  PRIMARY KEY (animal_ID),
                  FOREIGN KEY (animal_ID) REFERENCES Animal(animal_ID));
```

Treat(**Vet_ID**, **Animal_ID**, Number_of_Surgeries, Vaccinations)

```
CREATE TABLE Treat (vet_ID INTEGER,
                    animal_ID INTEGER,
                    number_of_surgeries INTEGER,
                    vaccination BOOLEAN,
                    PRIMARY KEY (vet_ID, animal_ID),
                    FOREIGN KEY (vet_ID) REFERENCES Vet(vet_ID),
                    FOREIGN KEY (animal_ID) REFERENCES Animal(animal_ID));
```

Adopter (Adopter_ID, Name, Age, Address, Criminal_Record)

```
CREATE TABLE Adopter (adopter_ID INTEGER,
                      name VARCHAR(20),
                      age INTEGER,
                      address VARCHAR(50),
                      criminal_record VARCHAR(20),
                      PRIMARY KEY (adopter_ID));
```

8. INSERT Statements:

```
INSERT INTO Donor (donor_ID, name, donated_item)
VALUES
(1, 'Macy Raymond', 'money'),
(2, 'Jack Shepherd', 'dog toys'),
(3, 'Lilah Knapp', 'cat toys'),
(4, 'Noah Lucero', 'money'),
(5, 'Angela Owens', 'cleaning supplies'),
(6, 'Jaden Marsh', 'money');
```

```
INSERT INTO Donate (donor_ID, branch_city, branch_province, amount)
VALUES
(1, 'Calgary', 'Alberta', 50),
(1, 'Vancouver', 'British Columbia', 45),
(2, 'Calgary', 'Alberta', 2),
(3, 'Burnaby', 'British Columbia', 4),
(4, 'Winnipeg', 'Manitoba', 70),
(5, 'Toronto', 'Ontario', 3),
(6, 'Ottawa', 'Ontario', 150);
```

```
INSERT INTO Branch (city, province, address)
VALUES
('Calgary', 'Alberta', '4706  Maynard Rd'),
('Vancouver', 'British Columbia', '84 Robson St'),
('Burnaby', 'British Columbia', '397 James Street'),
('Winnipeg', 'Manitoba', '4046 St Marys Rd'),
('Toronto', 'Ontario', '644 Danforth Avenue'),
('Ottawa', 'Ontario', '819 MacLaren Street');
```

```sql
INSERT INTO Volunteer (volunteer_ID, name, role, started_date, branch_city,
branch_Province)
VALUES
(37927, 'Joyce Page', 'Animal Care Assistant', '2024-04-24', 'Burnaby',
'British Columbia'),
(37940, 'Kian Wu', 'Animal Care Assistant', '2023-09-13', 'Calgary',
'Alberta'),
(41338, 'Jasmine McCann', 'Dog Walker/Cat Socializer', '2023-04-22',
'Winnipeg', 'Manitoba'),
(28387, 'Lucy Reid', 'Adoption Event Assistant', '2022-02-07', 'Calgary',
'Alberta'),
(38429, 'Denise Kim', 'Transport Volunteer', '2024-03-10', 'Calgary',
'Alberta');
```

```sql
INSERT INTO Role (role, hours_per_week)
VALUES
('Animal Care Assistant', 5),
('Dog Walker/Cat Socializer', 4),
('Adoption Event Assistant', 5),
('Shelter Administrative Support', 6),
('Transport Volunteer', 2),
('Photography/Videography Volunteer', 3);
```

```sql
INSERT INTO Vet (vet_ID, name, clinic)
VALUES
(1, 'Katelyn Lyons', 'Pawsitive Care Veterinary Clinic'),
(2, 'Barbara Bowen', 'Healing Paws Animal Hospital'),
(3, 'Revor Stevenson', 'Compassionate Creatures Veterinary Clinic'),
(4, 'Abby Benson', 'Pawsitive Care Veterinary Clinic'),
(5, 'Vanessa Wu', 'The Pet Palette Veterinary Clinic');
```

```sql
INSERT INTO WorkFor (vet_ID, branch_city, branch_province, hourly_rate)
VALUES
(1, 'Vancouver', 'British Columbia', 65),
(1, 'Burnaby', 'British Columbia', 50),
(2, 'Calgary', 'Alberta', 47),
(3, 'Winnipeg', 'Manitoba', 43),
(4, 'Vancouver', 'British Columbia', 70),
(5, 'Ottawa', 'Ontario', 59);
```

```sql
INSERT INTO Supplies (name, branch_city, branch_province, quantity)
VALUES
('Cleaning supplies', 'Calgary', 'Alberta', 10),
('Toys/Enrichment items', 'Vancouver', 'British Columbia', 40),
('Blankets', 'Burnaby', 'British Columbia', 25),
('Cleaning supplies', 'Burnaby', 'British Columbia', 14),
('Blankets', 'Winnipeg', 'Manitoba', 33),
('Toys/Enrichment items', 'Ottawa', 'Ontario', 47),
```

```sql
INSERT INTO EmployeeManages (employee_ID, branch_city, branch_province,
name, position, started_date)
VALUES
(1234, 'Calgary', 'Alberta', 'Evan Turner', 'Manager', '2022-01-15'),
(5678, 'Vancouver', 'British Columbia', 'Olivia Mason', 'Assistant Manager',
'2023-03-12'),
(9101, 'Burnaby', 'British Columbia', 'Sophia Carter', 'Veterinarian',
'2022-09-25'),
(1121, 'Winnipeg', 'Manitoba', 'Liam Johnson', 'Assistant Manager', '2023-
05-18'),
(3141, 'Ottawa', 'Ontario', 'Ella Brown', 'Animal Trainer', '2021-07-04');
```

```sql
INSERT INTO Position (position, salary)
VALUES
('Manager', 70000),
('Assistant Manager', 55000),
('Veterinarian', 80000),
('Shelter Assistant', 40000),
('Animal Trainer', 45000);
```

```sql
INSERT INTO Cage (cage_number, branch_city, branch_province, floor_number)
VALUES
(1, 'Calgary', 'Alberta', 2),
(2, 'Vancouver', 'British Columbia', 1),
(3, 'Burnaby', 'British Columbia', 3),
(4, 'Winnipeg', 'Manitoba', 1),
(5, 'Ottawa', 'Ontario', 2);
```

```sql
INSERT INTO FloorNumber (floor_number, size)
VALUES
(1, 'Large'),
(2, 'Medium'),
(3, 'Small'),
(4, 'Large'),
(5, 'Medium');
```

```sql
INSERT INTO Animal (animal_ID, name, breed, age, admission_date,
adoption_date, adoper_ID, cage_number, branch_city, branch_province)
VALUES
(1001, 'Max', 'Labrador', 4, '2023-07-12', NULL, NULL, 1, 'Calgary',
'Alberta'),
(1002, 'Bella', 'German Shepherd', 3, '2022-12-05', '2023-10-10', 3001, 2,
'Vancouver', 'British Columbia'),
(1003, 'Milo', 'Beagle', 2, '2023-04-22', NULL, NULL, 3, 'Burnaby', 'British
Columbia'),
(2001, 'Luna', 'Persian', 1, '2023-08-01', NULL, NULL, 4, 'Winnipeg',
'Manitoba'),
(1005, 'Charlie', 'Poodle', 5, '2021-09-15', '2023-01-20', 3002, 5,
'Ottawa', 'Ontario');
```

```sql
INSERT INTO AdoptionFee (breed, age, adoption_fee)
VALUES
('Labrador', 4, 400),
('German Shepherd', 3, 600),
('Beagle', 2, 650),
('Persian', 1, 500),
('Poodle', 5, 560);
```

```sql
INSERT INTO Dog (animal_ID, guarding_behavior)
VALUES
(1001, 'High'),
(1002, 'Moderate'),
(1003, 'Low'),
(1004, 'Low')
(1005, 'Moderate');
```

```sql
INSERT INTO Cat (animal_ID, napping_habits)
VALUES
(2001, 'Frequent napper'),
(2002, 'Sleeps after meals'),
(2003, 'No regular naps'),
(2004, 'Takes long naps'),
(2005, 'Short naps throughout the day');
```

```sql
INSERT INTO Adopter (adopter_ID, name, age, address, criminal_record)
VALUES
(3001, 'John Doe', 35, '123 Elm St, Vancouver, BC', 'None'),
(3002, 'Jane Smith', 28, '456 Maple Ave, Ottawa, ON', 'None'),
(3003, 'Michael Brown', 40, '789 Oak Rd, Calgary, AB', 'None'),
(3004, 'Emily Davis', 32, '321 Pine St, Burnaby, BC', 'None'),
(3005, 'Daniel Lee', 45, '654 Cedar Ln, Winnipeg, MB', 'None');
```